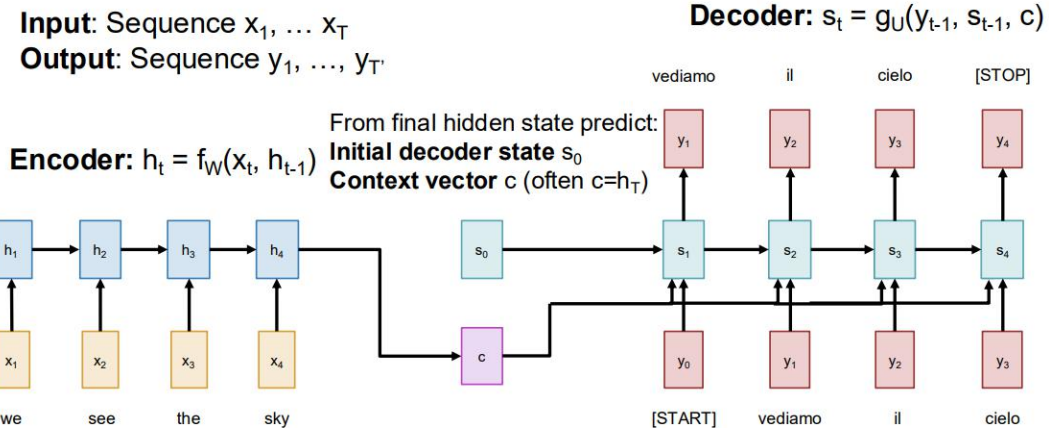
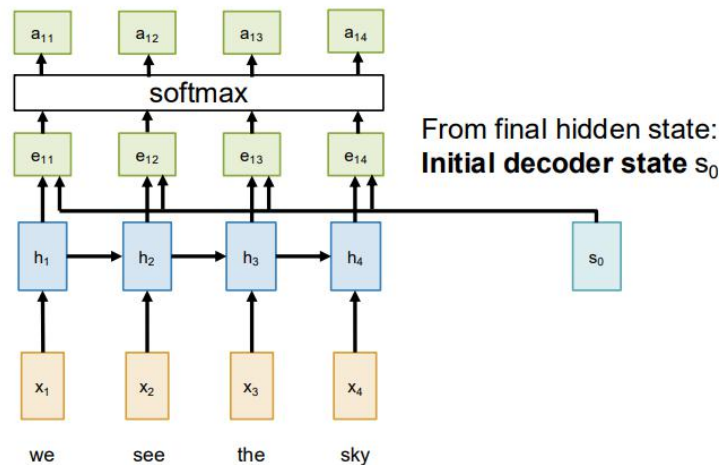


1. Vấn đề của RNN



Vấn đề nằm tại “c”: nếu như câu ngắn tầm 100 kí tự thì c có thể bao quát hết. Tuy nhiên nếu câu dài vd hơn 1024 kí tự thì không thể encode hết vào c được
 -> Solution: Mỗi lần sinh ra output thì nhìn lại hết các input tức là sinh ra y_2 thì nhìn x_1, x_2

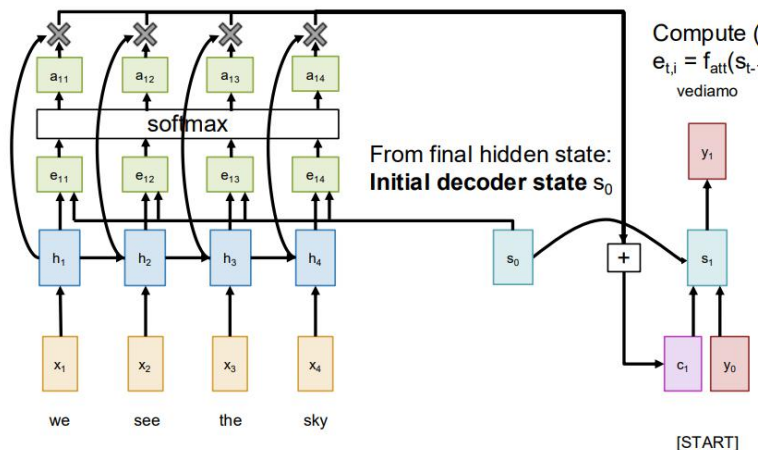
2. RNN và Attention



Alignment score: $e_{t,i} = f_{att}(s_{t-1}, h_i)$ với f_{att} là một Linear Layer

Mục tiêu: Đo mức độ liên quan giữa decoder state và encoder state thứ i (trong hình đang so s_0 lần lượt với h_1, h_2, h_3, h_4)

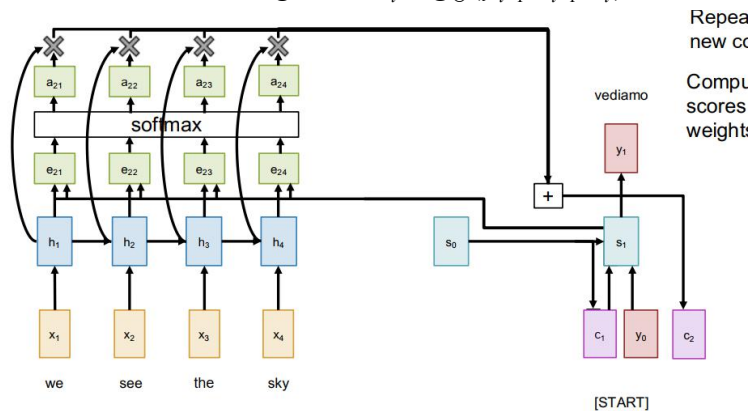
Sau đó, ta tính attention weight bằng cách sử dụng softmax function -> coi sự đóng góp của mỗi encoder so với decoder state



Context vector: $c_t = \sum_i a_{t,i} h_i$

Cái này cực kì quan trọng vì nó quyết định vì chỉ có $a_{t,i}$ là có thể cập nhật được sau mỗi decoder state, tức là với mỗi decoder state thì nó sẽ học các từ mà nó thấy quan trọng (vd vediamo a11 và a12 sẽ có trọng số quan trọng hơn 2 cái còn lại)

Decoder state mới sẽ được tính bằng cách $s_t = g_{\cup}(y_{t-1}, s_{t-1}, c_t)$

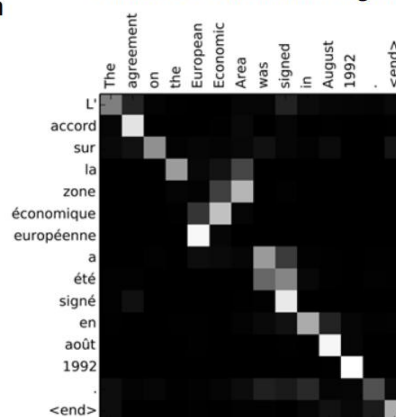


Sau khi có s_1 thì ta sẽ tiếp tục dùng s_1 để tính alignment score mới
 Điểm cần training cho mô hình học ở đây là hàm fatt -> sinh xác suất sao để dự đoán được tỉ lệ các từ cần thiết cho decoder state tiếp cao -> alignment score chuẩn
 -> Mục tiêu là làm sao để hàm fatt có thể dự đoán được chính xác xác suất cho từ tiếp theo

Note: “we see” = “vediamo” nên có thể alignment score là $a_{11} = a_{12} = 0.450$ trong khi 2 thằng còn lại tổng bằng 0.1 -> trường hợp thường xảy ra trong dịch ngôn ngữ

Example: English to French translation

Visualize attention weights $a_{t,i}$



Ví dụ về phân bố weight tương ứng -> Mục tiêu cần train đc

3. Attention layer

Thông qua phần trên ta rút ra được

3.1. Input

- Query vector $q[D_Q]$

- Data vectors $X[N_x \times D_Q]$

N_x là số lượng data vector và hay còn là chiều dài chuỗi input (h_1, h_2, h_3, h_4)

D_Q là chiều của vector ví dụ tại h_1 có 4 chiều

3.2. Output và Computation

Similarities (alignment score) công thức giống ở trên

Attention weight giống ở trên với shape $[N_x]$

Output vector: $y = \sum_i a_i X_i$

Với shape là $[D_Q]$

-> Xong lại vòng lại từ Similarities

Thường công thức của fatt là $e_i = q \cdot X_i / \sqrt{D_Q}$ -> tại sao phải chia cho $\sqrt{D_Q}$?

Bởi vì nếu như để số quá lớn như 9999999999 hay 1000000000000 thì qua softmax nó sẽ coi như bằng nhau hoặc sẽ có vài số có thể bị cho bằng 0 luôn -> vanishing gradient

*** Change: nếu là multiple query vector**

Query vector $Q[N_Q \times D_Q]$

Computation sẽ được tính theo công thức dưới

Similarities: $E = QX^T / \sqrt{D_X}$ $[N_Q \times N_X]$

$$E_{ij} = Q_i \cdot X_j / \sqrt{D_X}$$

Attention weights: $A = \text{softmax}(E, \text{dim}=1)$ $[N_Q \times N_X]$

Output vector: $Y = AX$ $[N_Q \times D_X]$

$$Y_i = \sum_j A_{ij} X_j$$

***Change: Seperate key and value (Ở đây sẽ không còn là RNN nữa nên dẹp sang 1 bên)**

Query vector $Q[N_Q \times D_Q]$

Data vector $X[N_X \times D_X]$

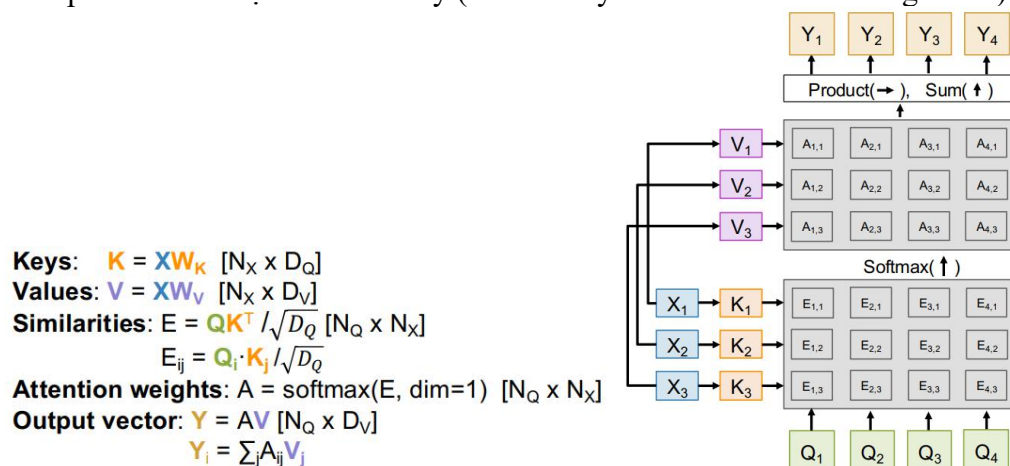
Key matrix $W_K[D_X \times D_Q]$

Value matrix $W_V[D_X \times D_V]$

-> Tại sao có key và có value matrix?

Nếu như lúc đầu chỉ có 1 cái X thì X vừa dùng để truyền dữ liệu và để so sánh alignment score -> vậy giờ tách ra 2 cái key matrix chỉ để so sánh tính similarity và value matrix để truyền dữ liệu

Computation sẽ được tính như này (kết cấu này khác nên bỏ RNN sang 1 bên)



Query vector có thể khác context vector(của RNN ở trên) vì query vector ở đây có thể tạo bằng nhiều cách khác nhau không bắt buộc là trạng thái của decoder như RNN.

Tuy nhiên chức năng vẫn giống nhau là để so sánh similarity và tính alignment score. Output vector mang ý nghĩa là representation mới của mỗi token sau khi đã nhìn toàn bộ các token khác.

-> Đây gọi là **Cross-Attention layer**

4. Self-attention layer

Khác với cross-attention layer thì input sẽ không Query vector mà là Query matrix.
Tức là trong khi cross-attention có Q và K,V là 2 nguồn khác nhau thì self-attention đều xuất phát từ chung 1 nguồn X -> self

Input:

Data vector $X[N \times D_{in}]$

Key matrix $W_K[D_{in} \times D_{out}]$

Value matrix $W_V[D_{in} \times D_{out}]$

Query matrix $W_Q[D_{in} \times D_{out}]$

Computation:

Queries: $Q = XW_Q [N \times D_{out}]$

Keys: $K = XW_K [N \times D_{out}]$

Values: $V = XW_V [N \times D_{out}]$

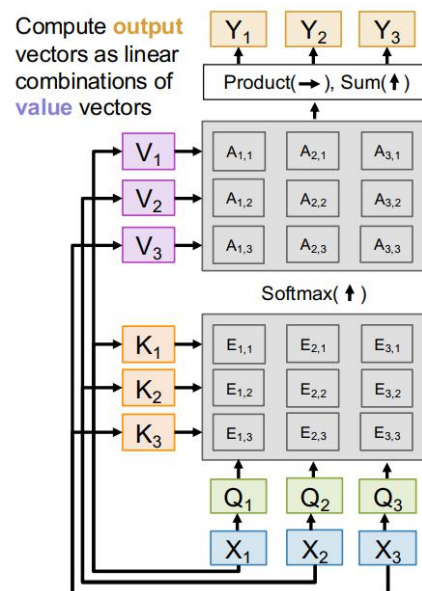
Similarities: $E = QK^T / \sqrt{D_Q} [N \times N]$

$$E_{ij} = Q_i \cdot K_j / \sqrt{D_Q}$$

Attention weights: $A = \text{softmax}(E, \text{dim}=1) [N \times N]$

Output vector: $Y = AV [N \times D_{out}]$

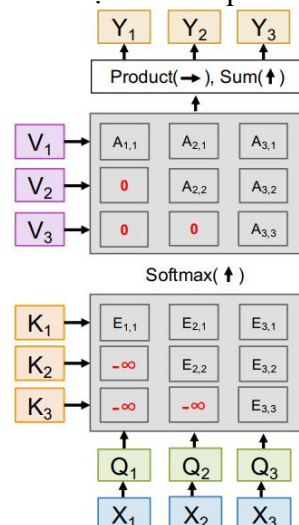
$$Y_i = \sum_j A_{ij} V_j$$



Note: Self-attention không care tới thứ tự của input dù bạn có permute cỡ nào (bạn shuffle input thì output shuffle y chang vậy) -> bởi vì đơn giản nó chỉ đang đo similarities của từng query với input thôi -> cách khắc phục là thêm 1 encoder position để fixed order input

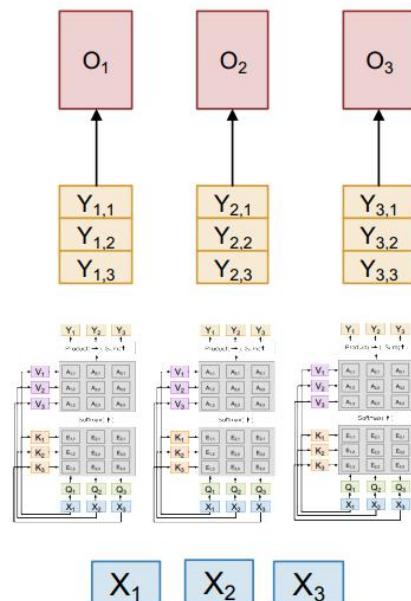
* Masked self-attention

Cái này sinh ra để tránh cho từ trước biết được từ tiếp theo là gì trong lúc train nên người ta mask các vị trí thành trừ vô cực để khi qua softmax nó thành 0



* Multi-head self-attention

Ta chạy nhiều self-attention song song, mỗi cái gọi là một head. Mỗi head giúp ta học mỗi quan hệ khác nhau (ngữ pháp, chủ-vị, nhân xưng,...)



Inputs:

Input vectors: $X [N \times D]$

Key matrix: $W_K [D \times HD_H]$

Value matrix: $W_V [D \times HD_H]$

Query matrix: $W_Q [D \times HD_H]$

Output matrix: $W_O [HD_H \times D]$

Computation:

Queries: $Q = XW_Q [H \times N \times D_H]$

Keys: $K = XW_K [H \times N \times D_H]$

Values: $V = XW_V [H \times N \times D_H]$

Similarities: $E = QK^T / \sqrt{D_Q} [H \times N \times N]$

Attention weights: $A = \text{softmax}(E, \text{dim}=2) [H \times N \times N]$

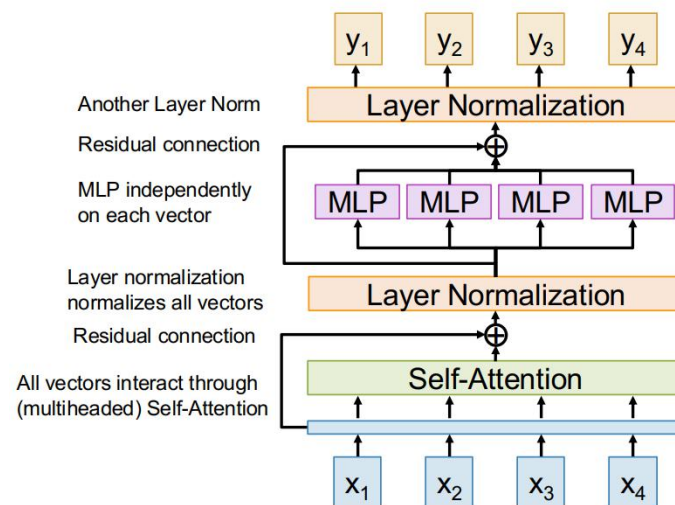
Head outputs: $Y = AV [H \times N \times D_H] \Rightarrow [N \times HD_H]$

Outputs: $O = YW_O [N \times D]$

O còn được gọi là Output projection fuses data from each head

Note: tại sao ko tăng các matrix lên mà lại phải tạo ra nhiều cái để chạy song song? -> Mô hình vẫn chỉ có 1 attention map và không phân tách được các loại quan hệ

5. Transformer block



Trong transformer block là chỉ có duy nhất ở self-attention thì các input có thể tương tác với nhau

Và transformer sẽ là các transformer block được kết nối với nhau

-> Kết luận: transformer là 1 neural network sử dụng self-attention ở mọi nơi